Mashbot — Requirement Scope and AI Assumptions

SE3002 Assignment 01 — Quality Evaluation of AI-Generated Software

Source SRS: *2010 - mashboot.pdf* — "Software Requirements Specification for Mashbot" (v1.1, 15 Feb 2010).

Per Assignment 01 rules: exactly 10 requirements, 7 Functional Requirements (70%) and 3 Non-Functional Requirements (30%), original requirement wording/IDs preserved where possible, no more than 3 of the 7 FRs are pure CRUD.

Part 1 — Functional Requirements (7)

Req. ID	Type	Requirement (brief)	Why selected / risk	AI assumption	Defence / basis
0240 / 0330 / 0380	FR (CRUD)	**Manage User Account** — create (0240), modify (0330), deactivate (0380) a user account. “New user accounts can be created”; “The system allows users to modify their accounts once created”; “The system allows user accounts to be deactivated.”	Foundational entity every other requirement depends on (auth, ownership, roles). Risk: deactivation vs. deletion semantics (SRS 0400/0410 ties deletion to absence of history) are easy to get wrong.	Deactivation is modeled as a status flag (active/deactivated) rather than physical deletion, and delete is only exposed when a profile has no owned campaigns — this satisfies 0400/0410 without a full audit-history subsystem.	Design decision (SRS 0400/0410 imply the rule; the flag-based mechanism is our implementation choice)
0480–0540	FR (CRUD)	**Manage Campaign** — a campaign has a name, pieces of content, a schedule, and user/group permissions; supports create/view/edit/delete.	Central entity of the whole product; combining name+content+schedule+permissions into one create/edit form is the main design risk (large surface area for defects).	Per-campaign granular permissions (0520) are simplified to the global account role (Contributor/Approver/Publisher) applied to every campaign the user can see, rather than a separate per-campaign ACL table.	Unsupported (SRS 0520 implies per-campaign permission assignment; we did not implement a separate ACL — documented limitation for MVP)
0590 / 0600 / 0610	FR (CRUD)	**Manage External Service Account** — associate a Mashbot account with an external social-network account; authenticate to it; interact with it through a standardized interface.	Real OAuth integration with Facebook/Twitter/etc. is out of scope for a graded MVP; risk is that a “mock” connection could be mistaken for a real integration.	External accounts are stored as a provider name + external username the user enters manually (“simulated” credential), not a live OAuth handshake. Clearly labeled in the UI as simulated.	Justified by explicit design decision (documented limitation, not hidden)
0140–0230	FR (non-CRUD)	**Contributor/Approver Content Workflow** — Contributors create/edit/delete content and submit it for approval; Approvers approve actions performed by Contributors; a user may hold more than one role.	Genuine business-rule/workflow behaviour (state machine + permission check), not CRUD — required to keep the FR set within the ≤3-CRUD limit.	Roles are stored as an array on the profile (contributor, approver, publisher) rather than a many-to-many “roles per product” table (0200), since the MVP has a single product (Campaigns).	Justified by explicit design decision (SRS 0200 scoped down for MVP)
0530 / 0550–0580	FR (non-CRUD)	**Content Scheduling** — a schedule maps times to publishing actions; content dragged onto a calendar defaults to a 12:00 AM go-live time unless the user sets one explicitly.	Combines a business rule (default go-live time) with a date/time boundary — good source of boundary test cases (midnight default, past-dated schedule rejection).	Drag-and-drop calendar (SRS Fig. 3) is simplified to a date/time picker on each content item; the “defaults to 12:00 AM” business rule is preserved exactly.	Justified by explicit design decision (interaction simplified, business rule preserved from SRS 0530)

Req. ID	Type	Requirement (brief)	Why selected / risk	AI assumption	Defence / basis
0650 / 0660 / 0670 / 0680	FR (non-CRUD)	**Login & Account-Validity Enforcement** — only a valid (not disabled/deleted) user may log in; login requires a valid username+password; the system uses the configured authentication module; sessions auto-expire after a configurable timeout.	Directly testable validation logic with clear pass/fail boundaries (deactivated account, wrong password, expired session).	"Configured authentication module" (0670) is fixed to Supabase Auth (email + password) for the MVP; no pluggable-auth abstraction was built. Timeout (0680) is a fixed constant rather than an admin-configurable setting.	Unsupported for the configurability part (0670/0680 call for admin-configurable behaviour; MVP hardcodes it) — documented limitation
0450 / 0460 / 0470	FR (non-CRUD)	**Self-Service Password Reset** — the system lets users reset their password; only a user's own password may be changed by that user.	Small but meaningful authorization rule (a user must not be able to change another user's password from the client) — good negative/invalid-input test target.	Password reset uses Supabase's built-in email reset-link flow rather than a custom SMTP integration (see NFR discussion of the Email System interface, SRS 0100–0130).	Justified by explicit design decision

CRUD count check: 0240/0330/0380 (Manage User Account), 0480–0540 (Manage Campaign), and 0590/0600/0610 (Manage External Service Account) = **3 of 7 FRs are pure CRUD**, at the assignment's limit. The remaining 4 FRs are workflow/business-rule/validation behaviour.

Part 1 — Non-Functional Requirements (3)

Req. ID	Type	Requirement (brief)	Why selected / risk	AI assumption	Defence / basis
0620	NFR (Security)	"The system encrypts data over a direct connection between the web client and the server."	Directly checkable via SonarQube security hotspots and by inspecting deployment TLS config; a common real-world regression (mixed content, plaintext cookies).	TLS termination is delegated entirely to the hosting platform (e.g. Vercel) and Supabase's own TLS-only endpoints; no custom TLS code is written or audited by us.	Supported by SRS; implementation is a platform configuration, not application code — documented limitation for SonarQube-based evaluation
0630	NFR (Availability / Backup)	"The system allows for the backup of the data system... Incremental backups should not create outages and full backups do not interfere with user interaction for more than 10 minutes."	Forces an explicit, defensible statement of what evidence can and cannot be produced for a hosted managed database — a good example of reporting a limitation instead of inventing a threshold.	Backups are delegated to Supabase's managed Postgres backup service; the "≤10 minutes of interference" threshold cannot be independently measured because we do not control the backup infrastructure.	Unsupported / not independently verifiable — reported as a limitation rather than fabricated
SRS §2.1.6 (Memory Constraints)	NFR (Resource / Performance)	"The Mashbot server requires no greater than 1 gigabyte of RAM... The web client requires no more than... approximately 256 megabytes of RAM."	A concrete, testable resource ceiling — server-side is largely inherited from the serverless platform; client-side is measurable via browser devtools memory profiling.	Server memory is bound by the serverless function's platform limit (not separately verified); client memory is spot-checked via Chrome DevTools on the Dashboard/Campaign pages only, not exhaustively.	Justified by explicit design decision for scope, with the measurement limitation stated

Out of scope for this MVP (explicitly not implemented): keyword alerts / "Explore" trending-topics view, dashboard analytics graphs (clickthrough rate, page views, comment aggregation), bulk user actions, per-campaign granular permissions, admin-configurable authentication module/timeout, Publisher-initiated immediate publishing to live external networks, email verification on registration, audio/video content types. These trace to Priority 3/4 items in the SRS's Requirements Apportioning section, or to requirements not selected into the 10-requirement scope.

Part 2 — AI-Assisted Development Record

Tooling

The baseline implementation of this MVP was produced with Claude Code (Anthropic), an AI-assisted coding tool, working from the Mashbot SRS (*2010 - mashboot.pdf*) and the selected 10-requirement scope in Part 1 above. The pair is responsible for the resulting code, reviewed the generated implementation, and recorded the assumptions below as required by the assignment brief.

How to read this section

Each assumption below is classified per the assignment's required basis: **(a) SRS** — directly supported by the SRS text; **(b) Design decision** — not explicit in the SRS, but a defensible scoping choice made and documented; **(c) Unsupported** — a gap between the SRS and the implementation that is not hidden.

Architecture-wide assumptions

- **1. Auth provider fixed to Supabase Auth.** SRS 0670 calls for a configurable authentication module with an internal fallback. The MVP hardcodes Supabase email/password auth and does not implement a pluggable module system. *(c) Unsupported — documented limitation, not a hidden gap.*
- **2. Session timeout is a fixed constant, not an admin-configurable setting (SRS 0680).** *(c) Unsupported.*
- **3. Roles are global, not per-product/per-campaign.** SRS 0200 describes roles assignable “for individual products”; campaign-level permissions (0520) suggest per-campaign ACLs. The MVP stores one role set per user account, applied uniformly across all campaigns they can access. *(b) Design decision — reduces scope to fit an MVP timeline; a per-campaign permissions table is the natural next iteration.*
- **4. External service accounts are simulated, not live OAuth integrations.** Connecting Facebook/Twitter/etc. would require registering developer apps with each platform, which is outside the scope of a graded coursework MVP. The UI clearly labels connections as “simulated” and stores only a provider name + external username. *(b) Design decision, explicitly surfaced in the UI so it is never mistaken for a real integration.*
- **5. No real publishing side-effects.** “Publishing” a piece of content changes its status in our own database only; it does not call any external network's API. This keeps the MVP safe to demo without real credentials while still exercising the full approval -> schedule -> publish state machine. *(b) Design decision.*
- **6. Backups, TLS termination, and server memory ceilings are delegated to the hosting platform.** (Supabase managed Postgres + Vercel/Node hosting) and are not independently implemented or measured by application code. See the NFR table in Part 1 for the specific limitations this creates for evidence-gathering. *(a) SRS-supported requirement, (c) unsupported evidence — this gap is reported rather than a measurement being fabricated.*
- **7. Email verification on registration (SRS use case 6) is not implemented.** Supabase Auth supports it, but it was left disabled for the MVP so the pair can create test accounts quickly during test execution. *(b) Design decision — a real release would enable it.*
- **8. Content types are limited to Text and Image (SRS 0550/0560, Priority 1).** Audio and Video (0570/0580, Priority 3) are out of scope for the initial release per the SRS's own priority scheme. *(a) SRS-supported (Priority 3 items are not expected in the initial release).*
- **9. Deleting a user account removes only the profiles row, not the underlying Supabase Auth user.** A full delete requires the service-role key, which the client-side/server-action code in this MVP intentionally never holds (keeping it out of the codebase avoids a severe SonarQube security hotspot for a secret key with database-wide privileges). *(b) Design decision, documented so it is not mistaken for a complete deletion.*

Baseline freeze

The first complete, runnable commit implementing all 10 selected requirements (git tag `baseline-v1`) is the baseline submitted to SonarQube and used for test execution in Part 3 of the assignment. Any defect fixes made after that point are committed separately so the evaluated baseline remains reconstructible from git history.